



App Fidélité Locale

Dossier UX/UI — Étude de cas complète : du problème au produit, de la recherche au prototype.

ALES CINO · UI/UX DESIGN · 2026

15 sections

01	Contexte & Problématique	p. 3
02	Objectifs & Positionnement	p. 4
03	Recherche utilisateur	p. 5
04	Benchmarking	p. 6
05	Userflow détaillé	p. 7
06	Démarche	p. 8
07	Arborescence & Parcours	p. 9
08	Design System : Foundations	p. 10
09	Design System : Composants	p. 11
10	Design System : Inventaire complet	p. 12
11	Une décision clé : le pivot QR Code	p. 13
12	Accessibilité	p. 14
13	Prototype	p. 15
14	Positionnement commercial	p. 16
15	Bilan & Apprentissages	p. 17

Pourquoi ce projet ?

Les petits commerces indépendants — boulangeries, cafés, salons de coiffure, artisans — n'ont ni le temps ni les moyens de développer leur propre programme de fidélité digital. Résultat : ils perdent en fidélisation face aux grandes enseignes qui, elles, disposent de vraies applications de fidélité.

Le client, de son côté, se retrouve avec une pile de cartes de fidélité en papier, difficiles à suivre, faciles à perdre, et sans aucune visibilité sur ses points ou ses récompenses.

Le problème commerçant

Pas de solution de fidélité accessible, pas de données sur ses clients, pas d'outil pour les faire revenir.

Le problème client

Une carte de fidélité différente par commerce, papier, facilement perdue, aucun suivi centralisé.

Hypothèse de départ

Et si une seule application permettait à un client de cumuler des points chez **plusieurs commerces locaux différents**, sans que chaque commerçant ait à développer quoi que ce soit ?

Deux expériences, un seul produit

Le produit n'appartient à aucun commerce en particulier : c'est une plateforme mutualisée pensée pour accueillir n'importe quel commerce local — boulangerie, café, institut, artisan. L'expérience se découpe en deux profils distincts avec des besoins opposés.

Client

Découvrir, cumuler, échanger

- Découvrir des commerces partenaires
- Cumuler des points automatiquement
- Consulter ses cartes de fidélité
- Échanger ses points contre des récompenses
- Présenter son QR Code pour valider un achat

Commerçant

Gérer, scanner, suivre

- Scanner le QR Code d'un client
- Attribuer des points en un geste
- Gérer son programme de fidélité
- Suivre ses statistiques et son activité
- Gérer sa fiche commerce publique

Ce que le produit n'est pas

Pas de réseau social, pas de chat, pas de gamification excessive. Le principe directeur tout au long du projet : **simple, local, rapide, utile**. Chaque écran répond à un besoin réel — on n'ajoute pas d'écran juste pour gonfler le portfolio.

Des hypothèses construites, pas une enquête terrain

Précision méthodologique : contrairement à un projet avec client réel (où un questionnaire est diffusé à un vrai panel), ce projet est un exercice de portfolio sans utilisateurs à interroger. La recherche s'appuie donc sur des **données secondaires publiques** et des **hypothèses explicitement identifiées comme telles** — pas de statistiques inventées.

Donnée secondaire retenue

Selon le Baromètre Fidme 2024 (panel de 5 millions de consommateurs français), un foyer possède en moyenne **16 cartes de fidélité** et n'en utilise que **2 par semaine** en caisse.

Source : Fidme, "Pourquoi Fidme est la meilleure application pour vos cartes de fidélité", 2024.

Deux personas — hypothèses de travail

Persona Client

Camille, 34 ans

Hypothèse construite à partir du comportement d'achat local courant.

- Fréquente 4-5 commerces de son quartier régulièrement
- Possède des cartes papier qu'elle oublie ou perd souvent
- Veut de la simplicité — pas une appli de plus à configurer

Persona Commerçant

Karim, 41 ans, boulanger

Hypothèse construite à partir des contraintes connues des indépendants.

- Pas de budget ni de temps pour développer une appli maison
- Veut fidéliser sans complexifier le passage en caisse
- Se méfie des outils "trop numériques" à apprendre

Ce que ça implique pour le design

Si une personne jongle déjà avec 16 cartes, ajouter "encore une appli" n'a de sens que si l'onboarding est quasi instantané et si la valeur (centraliser plusieurs commerces) est visible dès les 10 premières secondes — d'où le choix d'un onboarding à un seul écran avant la création de compte.

Concurrent direct + inspirations hors secteur

Trois applications réelles ont été analysées : une concurrente directe sur le marché de la fidélité, et deux références hors-secteur choisies pour un principe UX précis à emprunter.

Fidme — concurrent direct (cartes de fidélité dématérialisées)

Ce qui fonctionne : centralisation de toutes les cartes en un seul endroit, ouverture rapide en caisse, 4 millions d'utilisateurs actifs.

Limite identifiée : Fidme numérise des cartes de fidélité qui **existent déjà** chez les enseignes. Elle ne résout pas le problème du petit commerçant qui n'a **aucun programme digital** à numériser — c'est exactement l'angle mort que ce projet vient combler.

Too Good To Go — découverte de commerces locaux

Principe emprunté : une app anti-gaspi avec un parcours ultra-simple en 4 étapes (découvrir → réserver → récupérer) et une géolocalisation automatique qui affiche directement les commerces à proximité sans que l'utilisateur ait à chercher une ville. Ce principe de **zéro friction géographique** a directement inspiré l'écran Découverte du projet.

Starbucks Rewards — mécanique QR + points

Principe emprunté : "scannez votre QR code en caisse, vos étoiles sont ajoutées automatiquement" — exactement le mécanisme retenu comme parcours principal de validation des points dans ce projet, plutôt que la saisie manuelle d'un code. Starbucks confirme aussi l'intérêt d'un seuil de récompense lisible (150 étoiles → boisson offerte), repris dans la logique "points requis" des cards de récompense.

Synthèse

Ce benchmark confirme une opportunité claire : aucune des trois références ne propose de **programme mutualisé pour petits commerces indépendants**. Fidme numérise l'existant, Too Good To Go traite l'anti-gaspi, Starbucks est propriétaire à une seule enseigne. Le positionnement du projet reste donc différenciant.

Le parcours le plus critique : valider des points

Contrairement à l'arborescence générale (linéaire), ce flow détaille les embranchements réels — y compris les cas d'échec — du parcours le plus utilisé de l'app, côté commerçant.



Ce niveau de détail (avec les branches d'échec) a directement déterminé la liste des écrans et overlays à concevoir : écran Scanner, écran Client identifié, overlay Succès, et 3 overlays d'erreur distincts — plutôt que de découvrir ces besoins une fois les écrans "propres" déjà dessinés.

Un process itératif, pas linéaire

Le projet n'a pas suivi un chemin figé du premier coup — la direction visuelle et même le parcours de validation des points ont été repensés en cours de route, sur la base de retours et d'une remise en question continue.



13 écrans de base — 41 au total avec les états

Chaque écran a été validé par rapport à un besoin utilisateur précis avant d'être dessiné — pas d'écran ajouté "pour faire joli". Les 13 écrans ci-dessous forment le squelette des deux parcours ; ils se déclinent ensuite en 41 écrans réels une fois les états (vide, erreur, chargement, confirmation) pris en compte — détail en section 13.

Parcours Client



Parcours Commerçant



Distinction clé : "Mes cartes" vs "Mes récompenses"

Mes cartes	Mes récompenses
Les programmes de fidélité auxquels le client participe — une carte par commerce, avec sa progression.	Les récompenses concrètes que le client peut échanger ou a déjà utilisées.

Une base de tokens, pas des valeurs à la main

Chaque couleur, taille et espacement provient d'une variable — jamais fixé manuellement écran par écran. Résultat : un changement de teinte de marque se répercute automatiquement sur tout le fichier.

Palette



Chaque couleur existe en 10 nuances (50→900), avec des modes Light/Dark, et des tokens fonctionnels (Background, Text Primary, Border...) qui pointent vers ces couleurs de base.

Typographie

Une seule famille : **Inter**. 13 Text Styles couvrent tous les besoins, de Display (32px) à Caption (12px) — jamais de taille fixée à la main.

Display / H1

Body / Label

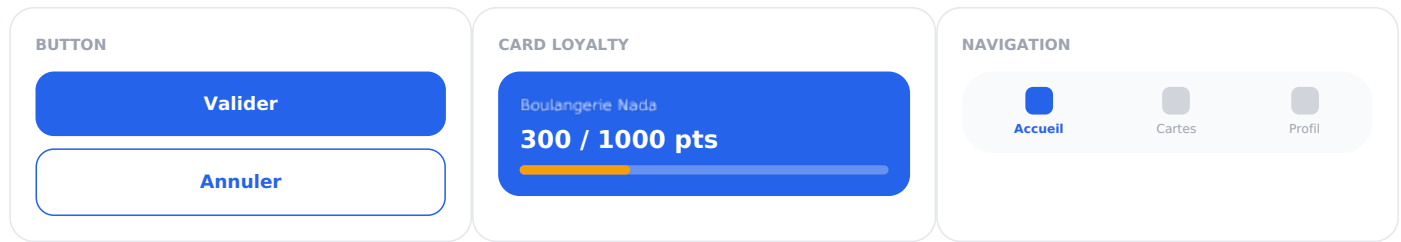
Spacing & Radius

Échelle unique, pas de valeurs arbitraires (13px, 27px...) :

4 · 8 · 12 · 16 · 20 · 24 · 32 · 40 · 48 · 64

Une bibliothèque réutilisable, pas des écrans isolés

Plus de 25 composants construits en Auto Layout avec variants (Button, Input, Card, Navigation, Modal, Toast, QR Code...), chacun avec ses états (Default, Focus, Error, Disabled...). Chaque écran est ensuite assemblé à partir de ces briques — jamais redessiné à la main.



Icônes — un vrai système, pas des emoji

19 icônes vectorielles dessinées sur mesure et nommées selon la convention **Lucide** (bibliothèque open-source), pour un transfert direct vers le code : `import { ChevronRight } from "lucide-react"` sans étape de conversion.

25+ composants, organisés par catégorie

Chaque composant existe une seule fois dans le fichier, avec ses variants — jamais dupliqué ou redessiné à la main sur un écran.

NAVIGATION & RECHERCHE

Navigation / Bottom (Client)

Navigation / Merchant

Search

Filter Chip

SAISIE & ACTIONS

Button (4 types × 3 tailles × 4 états)

Input / Text

Checkbox

Radio

Switch

Code Input / 4 Digits

Dropdown

Value Stepper

Date Picker

CARDS

Card / Business

Card / Loyalty

Card / Reward

Card / Statistics

FEEDBACK & ÉTATS

Badge

Toast (4 types)

Modal (4 types)

Empty State (7 variantes)

Spinner

Skeleton (4 variantes)

LISTES & AVATARS

List Item (4 types × 4 états)

Avatar (4 tailles)

SPÉCIFIQUE COMMERÇANT & QR

QR Code (4 états)

Points Configuration

Reward Configuration

Les graphiques (Line / Bar / Donut) affichés sur le Dashboard sont des illustrations statiques, pas encore des composants réutilisables — à transformer en vrai composant si le produit intègre un vrai outil de data visualisation.

ICÔNES

Icon — 19 variantes, nommées Lucide

Convention de nommage

Catégorie / Propriété / Valeur

Ex. Button/Primary/Large, Card/Business/Featured. Les variants suivent toujours Propriété=Valeur (ex. State=Focus). Interdits dans tout le fichier : "Rectangle 123", "Frame copie", "Test", "Button final 2" — chaque nom doit être compréhensible par un développeur qui n'a jamais ouvert le fichier.

Repenser le parcours le plus utilisé de l'app

La toute première version du parcours de validation des points reposait uniquement sur un code à 4 chiffres donné oralement par le commerçant. Fonctionnel, mais avec de la friction : le client doit écouter, mémoriser, taper.

Avant

Code à 4 chiffres uniquement

Le commerçant communique un code oralement, le client le tape sur son téléphone. Fonctionne, mais demande une interaction à deux et une saisie manuelle à chaque fois.

Après

QR Code comme parcours principal

Le client présente simplement son QR Code, le commerçant scanne. Le code à 4 chiffres reste disponible comme solution de secours (caméra en panne, problème réseau) — jamais supprimé.

Pourquoi ce n'est pas présenté comme une fonctionnalité "accessibilité"

Le QR Code n'a pas été positionné comme une option pour un public spécifique, mais comme une **réduction de friction pour tout le monde** : personne n'a besoin de chercher, lire ou taper un code. C'est un principe de design plus large que d'accessibilité pure — bien que ça profite particulièrement aux utilisateurs les moins à l'aise avec les interfaces numériques.

Un audit concret, pas une case à cocher

Un audit spécifique a été mené en se demandant : "cette app serait-elle compréhensible par une personne âgée, peu à l'aise avec le numérique ?" Plusieurs frictions réelles ont été identifiées et corrigées.

PAIRE DE COULEURS	RATIO CALCULÉ	VERDICT WCAG AA
Texte blanc sur bouton bleu (Primary)	5.17 : 1	Conforme
Texte principal (Neutral/900) sur fond blanc	17.74 : 1	Conforme
Texte secondaire / placeholder sur fond blanc	7.56 : 1	Conforme
Texte blanc sur fond d'erreur (rouge)	4.83 : 1	Conforme
Texte blanc sur badge orange (points)	2.15 : 1	Non conforme → corrigé*

*Un vrai défaut a été détecté et corrigé pendant l'audit : le texte blanc sur badge orange échouait largement le seuil AA. Le texte a été repassé en gris foncé sur ces badges (nouveau ratio : 8.26 : 1).

Zones tactiles $\geq 44 \times 44$ px partout. Aucune information transmise uniquement par la couleur (icône + texte systématiques pour les états).

Limite assumée

Le concept même de "scanner un QR Code" demande un minimum d'aisance avec un smartphone. La vraie réponse long-terme identifiée serait un mode "grand texte" activable plutôt que des ajustements ponctuels — un axe de suite pour une V2.

Un fichier cliquable, audité écran par écran

Chaque zone interactive (boutons, cards, items de navigation) est câblée avec de vraies transitions (Smart Animate), des overlays pour les modales de confirmation, et des liens de secours vers le code manuel. Les parcours Client et Commerçant sont indépendants et démontrables séparément.

Audit de robustesse

Une fois les 13 écrans principaux posés, chacun a été passé au crible pour vérifier qu'aucune action n'était sans suite : un bouton "Réessayer" qui ne fait rien, un écran sans retour possible, une action sensible sans confirmation. Cet audit a fait passer le fichier de 13 écrans "happy path" à **41 écrans réels**, et a mis au jour de vrais défauts corrigés dans la foulée :

DÉFAUT TROUVÉ	CORRECTION
6 boutons "Réessayer / Découvrir / Valider" sans destination	Câblés vers l'écran logique correspondant
4 écrans sans aucun moyen de revenir en arrière	Lien "← Retour" ajouté
Écran de chargement qui ne menait nulle part	Déclencheur "après délai" (1,2s) → transition automatique vers l'écran suivant
Déconnexion et échange de récompense en un seul tap	Étape de confirmation ajoutée (annulable) avant l'action
Bug de composant : 5 variantes d'État Vide non reliées à leurs textes	Corrigé à la source — répare tous les usages du fichier d'un coup

États couverts

- Succès (points ajoutés, récompense échangée, infos enregistrées)
- Erreurs (identifiants, QR invalide, déjà utilisé, réseau, client introuvable)
- Vides (aucune carte, aucun client, aucun résultat de recherche)
- Chargement (avec transition automatique) et expiration
- Confirmations avant action sensible (déconnexion, échange)

Ce que ça change pour le développement

- Composants nommés proprement (Button/Primary/Large...)
- Icônes mappées 1:1 sur une lib open-source réelle
- Descriptions d'accessibilité sur chaque composant clé
- Auto Layout partout — le contenu variable ne casse rien

Un produit, pas juste un exercice

Une landing page B2B a été construite pour présenter la valeur du produit aux commerçants — sans inventer de prix ni de fonctionnalité non conçue dans le reste du fichier.

"La fidélité client, simplement."

Une seule appli de fidélité pour tous les commerces locaux.

Pour les commerçants

- Fidélisation sans effort
- Statistiques en temps réel
- Fini les cartes papier

Pour les clients

- Une seule application
- Plusieurs commerces réunis
- Récompenses à débloquer

Ce que ce projet a demandé

9

pages de fichier Figma structurées

25+

composants réutilisables avec variants

41

écrans et états Client & Commerçant, câblés

Ce que je retiens

La difficulté n'était pas de faire "de jolis écrans", mais de tenir une **cohérence de système** sur la durée : une couleur changée quelque part doit se retrouver partout, un composant modifié doit rester fiable sur tous les écrans qui l'utilisent. C'est aussi ce qui a permis de remettre en question et corriger des décisions déjà prises (le pivot QR Code, la distinction Mes cartes / Mes récompenses) sans repartir de zéro.

Prochaine étape naturelle : coder une version fonctionnelle du prototype, en s'appuyant directement sur cette structure de composants et ce système d'icônes déjà pensés pour le développement.